Find Kth Largest Value In Bst

AlgoExpert · Medium · Binary Search Trees

The problem

- Reverse in-order traversal, stopping after k nodes.

Intuition

- A reverse in-order traversal visits values from largest to smallest, so stop at the kth.

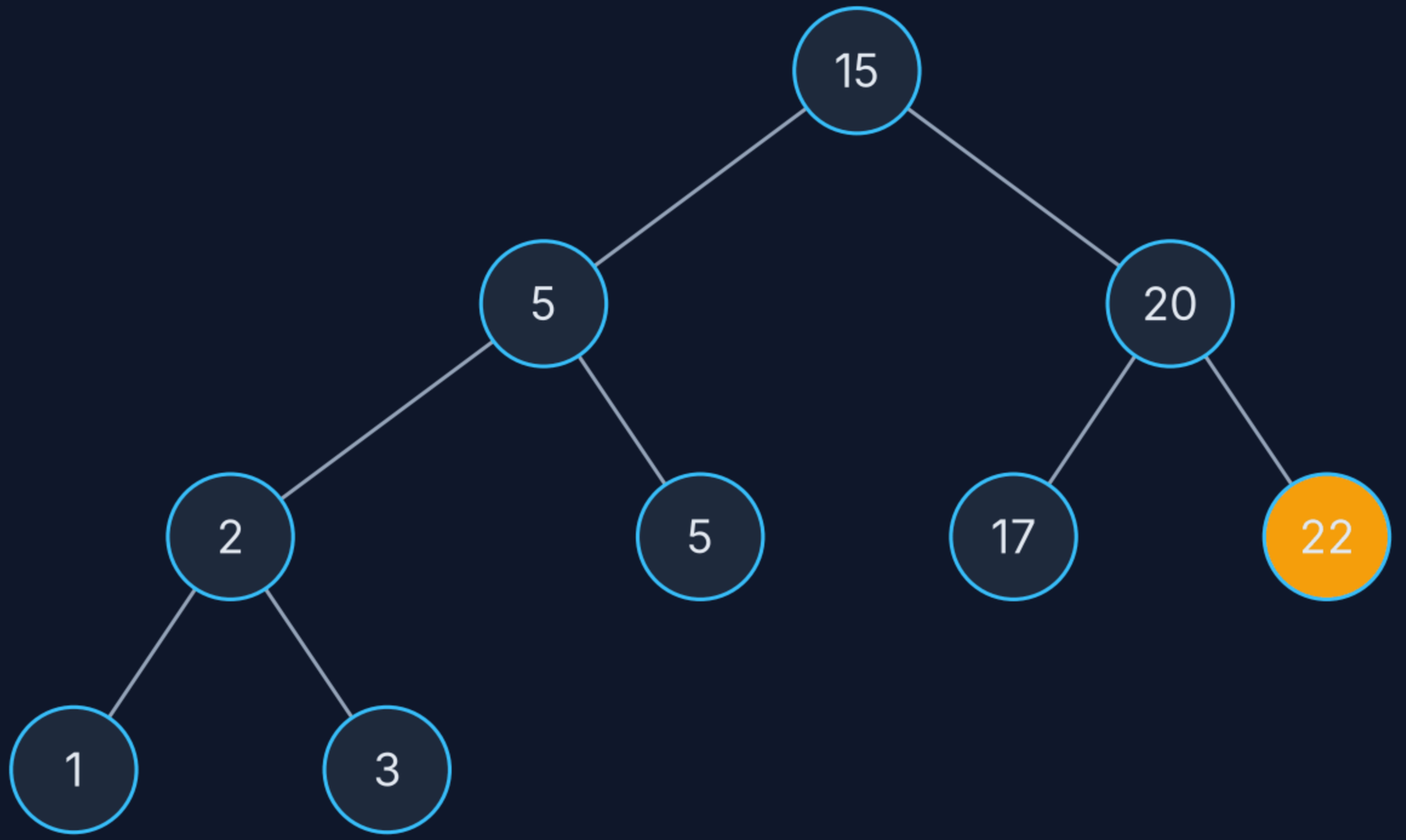
Brute force vs optimal

- In-order into a list, take list[n−k] → O(n) time and space
- Reverse in-order, stop at k → O(h + k)
- Only visits what's needed

Approach

- 1. Traverse right, node, left
- 2. Count visited nodes
- 3. Stop at the kth and return its value

Worked example

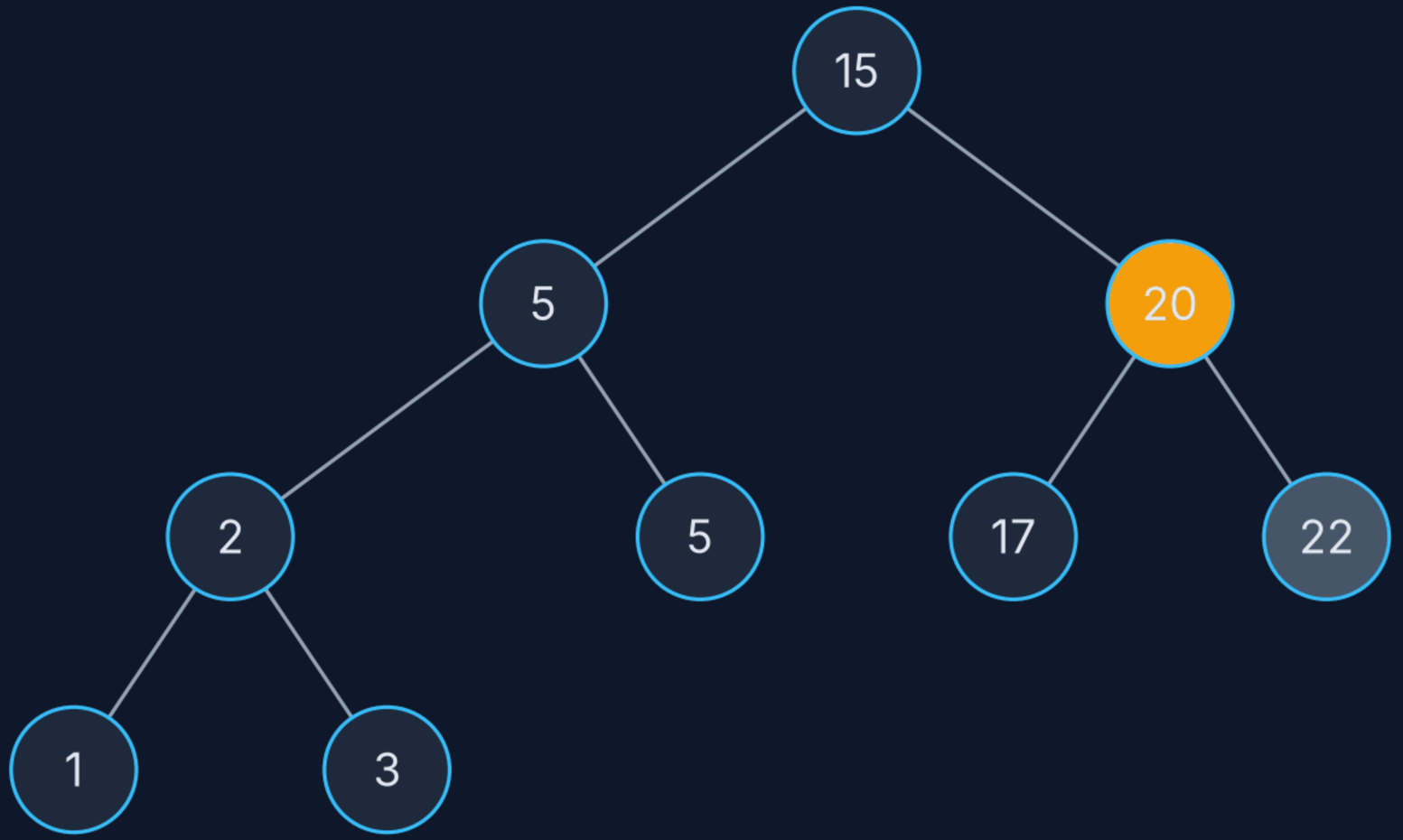



```
k = 3

visited: 22
```

Visit #1: 22

Worked example

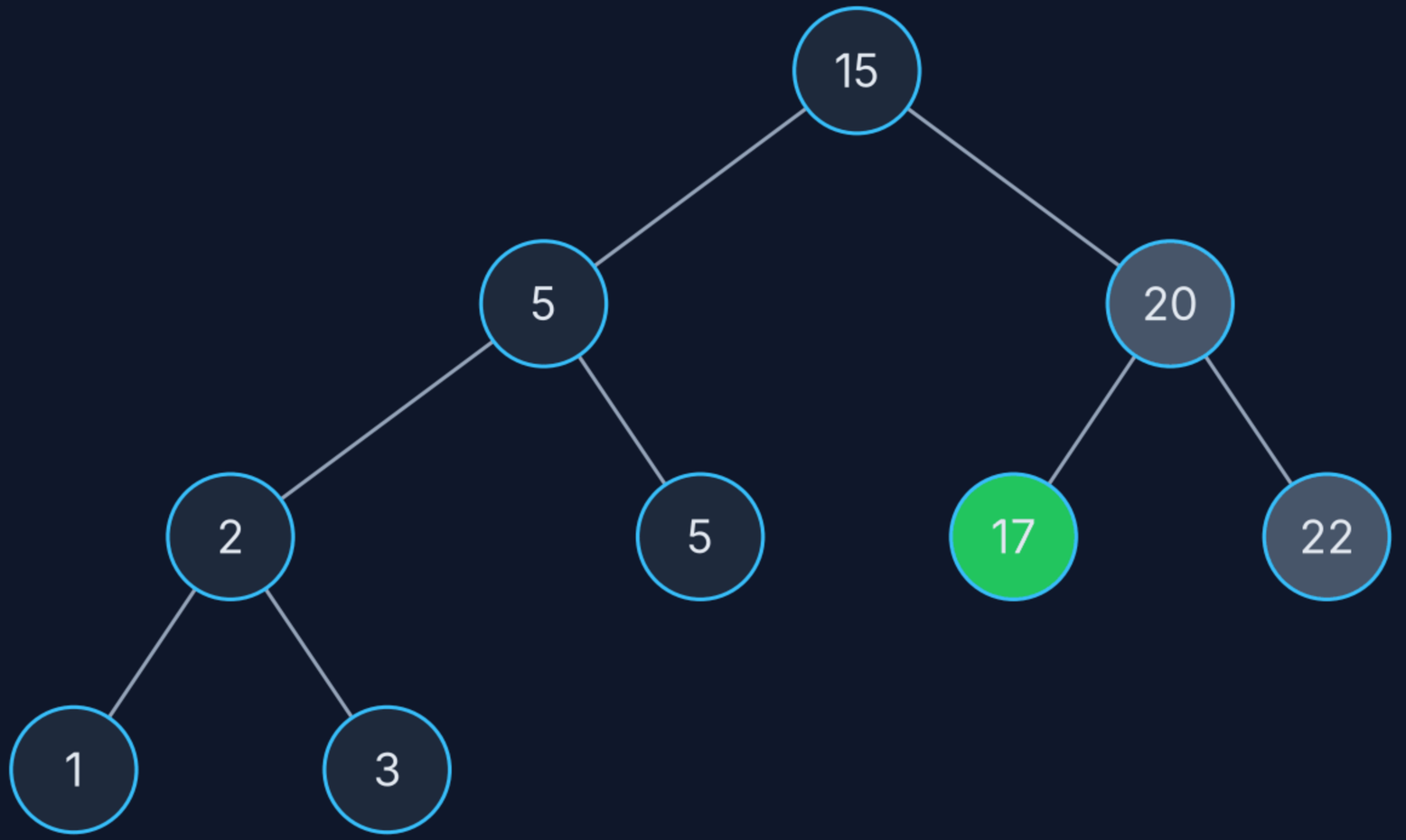



```
k = 3

visited: 22 20
```

Visit #2: 20

Worked example




```
k = 3

visited: 22 20 17
```

Visit #3: 17 ← k-th largest

Java solution

```java
public class FindKthLargestValueInBst {
    private static int seen, answer;

    public static synchronized int findKthLargestValueInBst(BST tree, int k) {
        seen = 0; answer = -1;
        walk(tree, k);
        return answer;
    }

    private static void walk(BST n, int k) {
        if (n == null || seen >= k) return;
        walk(n.right, k);
        if (seen < k) { seen++; if (seen == k) { answer = n.value; return; } walk(n.left, k); }
    }
}
```

Complexity

- Time: O(h + k)
- Space: O(h)

Edge cases & pitfalls

- Duplicates count separately (5, 5)
- K = 1 → the maximum
- Stop recursion once k values are seen

Interview tips & follow-ups

- Follow-up: many queries → augment nodes with subtree sizes
- Iterative reverse in-order with a stack

Recap

- A reverse in-order traversal visits values from largest to smallest, so stop at the kth.
- Complexity: O(h + k)